A3-PS003-TC534

Federated Traffic-Flow Prediction with Communication-Cost Evaluation

Puspam Adak
puspitaadak9876@gmail.com

AIORI Hackathon 3

October 13, 2026

Problem Description

- **High communication cost:** Centralized forecasting requires raw sensor data to be transmitted to a cloud server.
- **Privacy exposure:** Central collection concentrates sensitive travel-related data.
- **Heterogeneous traffic:** Different intersections have distinct traffic patterns, creating non-IID datasets.
- **Accuracy versus bandwidth:** Federated training must balance forecast quality against model-update transmission costs.
- **Convergence challenges:** Different local data distributions can slow or destabilize FedAvg convergence. [1][2]

Background

- Predict future traffic conditions from historical sensor readings. Reliable predictions can support congestion monitoring and smart-city traffic management.
- Centralized training collects data at one server. Federated learning keeps training data at intersections and exchanges model updates with a central coordinator. [1]
- IID clients have similarly distributed data; non-IID clients represent differing local traffic patterns. Evaluate both using chronological splits to avoid future information leaking into training.
- METR-LA is a practical candidate containing traffic-speed readings at five-minute intervals. [3][4]
- Sending smaller model updates instead of raw observations can reduce data movement, but it does not automatically guarantee lower total communication or formal privacy protection. Both must be evaluated appropriately. [1]

Literature & RFC Review

Reference	Key findings	Relevance
[1] McMahan et al. (2017), FedAvg	Introduces federated learning through local training and iterative model averaging; communication is a major constraint.	Core algorithm for our simulator.
[2] Li et al. (2020), On the Convergence of FedAvg on Non-IID Data	Studies how heterogeneous client data affects convergence and the trade-off between communication efficiency and convergence.	Justifies IID versus non-IID experiments.
[3] Li et al. (2018), Diffusion Convolutional Recurrent Neural Network (DCRNN)	Establishes a spatiotemporal approach to traffic forecasting using traffic-network structure and temporal dependencies.	Traffic forecasting baseline and dataset context.
[4] METR-LA / PEMS-BAY datasets	Public sensor-based traffic-speed datasets used extensively in forecasting research.	Provides reproducible experimental data.
[5] IETF RFC 8949 CBOR (2020)	Defines a compact binary representation intended to support small message sizes.	Optional implementation reference.

Proposed Solution

- Load METR-LA or another suitable public dataset, clean missing values, create chronological train/test splits, and partition training data across 3–5 clients.
- Create IID partitions and non-IID partitions that represent different traffic patterns across virtual intersections.
- Train a compact forecasting model centrally for the centralized baseline. For federated training, distribute global weights, train locally, return updates, and aggregate them using FedAvg.
- Measure MAE, RMSE, communication bytes per round, cumulative communication, and convergence. Repeat all configurations with three random seeds.
- A compact multilayer perceptron (MLP) using a fixed window of past traffic readings to predict the next reading. This keeps the prototype manageable and makes communication measurements easier to interpret.

Optimization Proposed

- Train models locally on 3–5 clients and aggregate their updates centrally to build a shared traffic-prediction model.
- Compare IID and non-IID data distributions to understand how differences between intersections affect prediction accuracy and convergence.
- Measure the bytes uploaded and downloaded during every training round and track cumulative communication costs.
- Choose a lightweight model such as an MLP to limit model-update size and keep computation manageable.
- Run each configuration with three random seeds and report MAE, RMSE, communication costs, and convergence behavior.
- Use chronological train/test splits so that future observations are not used during training.
- Optionally evaluate compact binary serialization to reduce communication overhead, measuring actual byte savings against the baseline.

Solution Architecture & Design

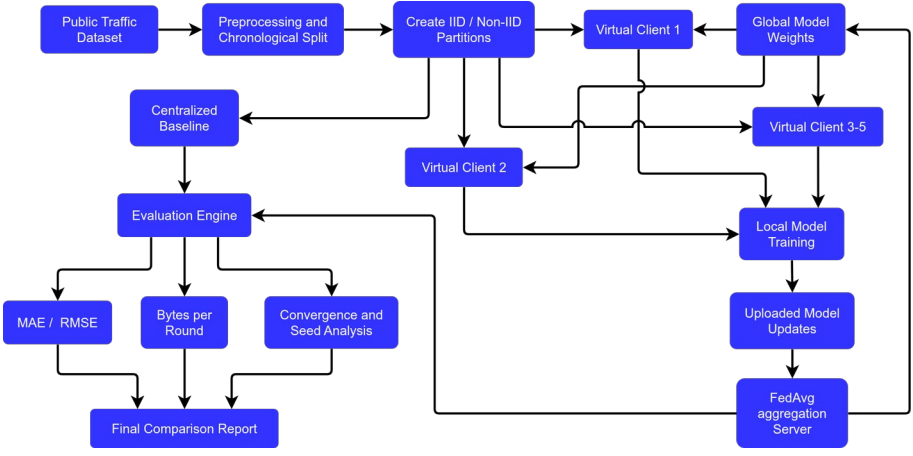


Figure: System Architecture UML Diagram

Data Flow Diagram

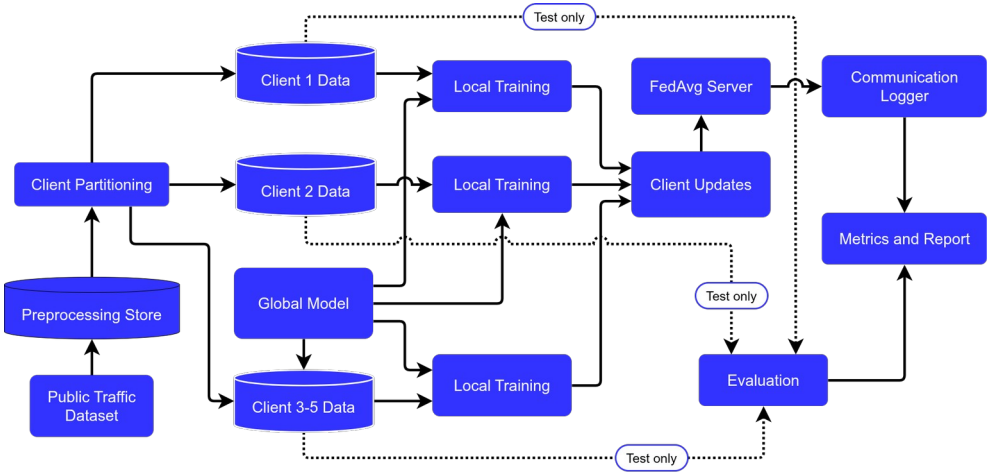


Figure: Data Flow Diagram (DFD)

Deployment Timeline

- **Days 1–2:** Dataset and setup: Create the private GitHub repository, configure the project environment, acquire the dataset, inspect missing values, and implement chronological splitting.
- **Days 3–4:** Centralized baseline: Implement the compact forecasting model, training loop, MAE/RMSE calculations, and saved evaluation results.
- **Days 5–6:** Federated implementation: Implement virtual clients, FedAvg aggregation, IID/non-IID partitions, and per-round byte accounting.
- **Days 7–8:** Experiments and optimization: Run all configurations using three random seeds. Compare forecast quality, cumulative bytes, and convergence. Test optional optimizations only if time permits.
- **Days 9–10:** Validation and demonstration: Verify reproducibility, finalize the comparison report and diagrams, prepare the pseudocode demonstration, and complete repository handover requirements.

Bibliography

[1] McMahan, B. et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS. Paper

[2] Li, X. et al. (2020). On the Convergence of FedAvg on Non-IID Data. ICLR. Paper

[3] Li, Y. et al. (2018). Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. ICLR. Paper

[4] Kwak, S. (2021). PEMS-BAY and METR-LA in CSV. Zenodo. Dataset

[5] Bormann, C. and Hoffman, P. (2020). RFC 8949: Concise Binary Object Representation (CBOR). IETF. RFC

Thank You for Your Attention!